

DineMatch

A Personalized Restaurant Recommendation Platform

SE/CS/CE 3354, Software Engineering, Deliverable #2

Fathima Bint-Khalid • Umar A. Hasan • Saumya Aditya Patel • David Huynh

Objective



**Most apps just show listings.
DineMatch ranks them for you.**

PERSONALIZED RANKING

Filters, history, bookmarks. Not just star count.

FOCUSED FEATURES

Search, accounts, reviews, bookmarks. Nothing extra.

REAL DATA

Sourced from Google Maps. We add our own “vibe” tags.

Core Idea

Pick cuisine, price, location, and vibe. Get a list built around YOUR history, not everyone else's.

- Search & filter restaurants
- Create account / log in
- Bookmark favorites
- Leave ratings & reviews
- Get personalized suggestions

Functional & Non-Functional Requirements

FUNCTIONAL

- 1 Filter restaurants by cuisine, price, location, vibe
- 2 Rank restaurants by relevance score
- 3 Create account & log in
- 4 Bookmark restaurants and view saved list
- 5 Leave star rating + written review
- 6 Embedded Google Maps view in results

NON-FUNCTIONAL

Performance: search loads under 3 seconds

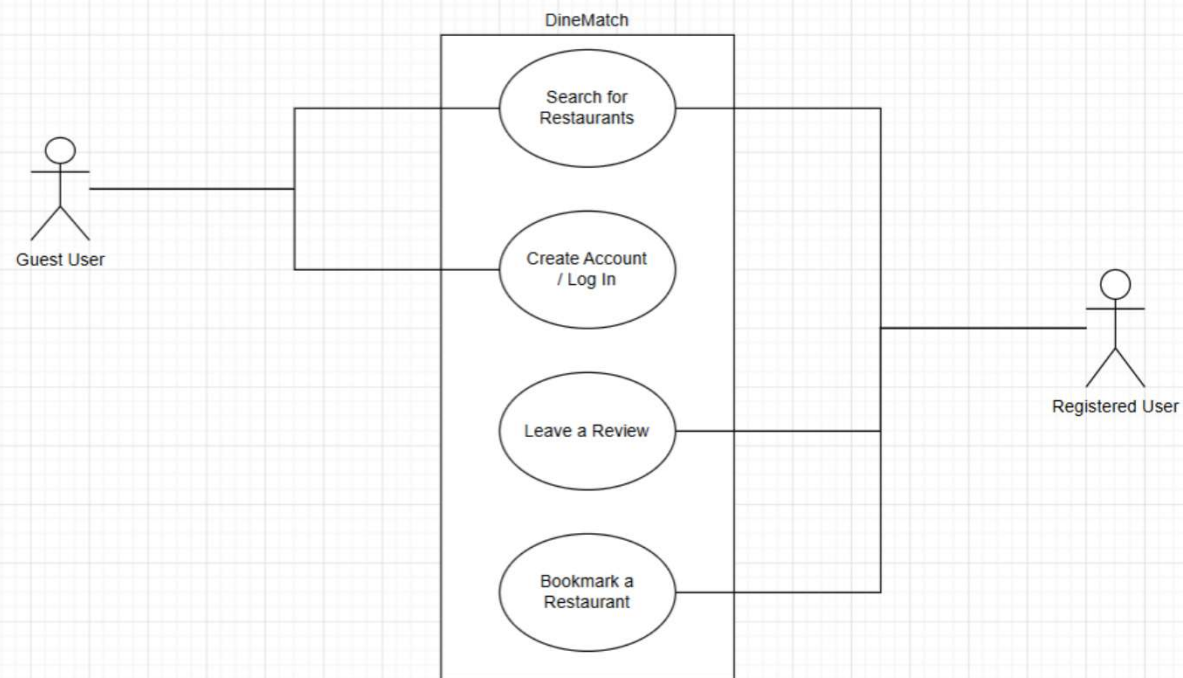
Security: hashed passwords, 24-hour token expiry

Dependability: 99% uptime at peak hours

Usability: no tutorial needed for new users

Legislative: users opt in before data is used for recommendations

Use Case Diagram



Actors

Guest User

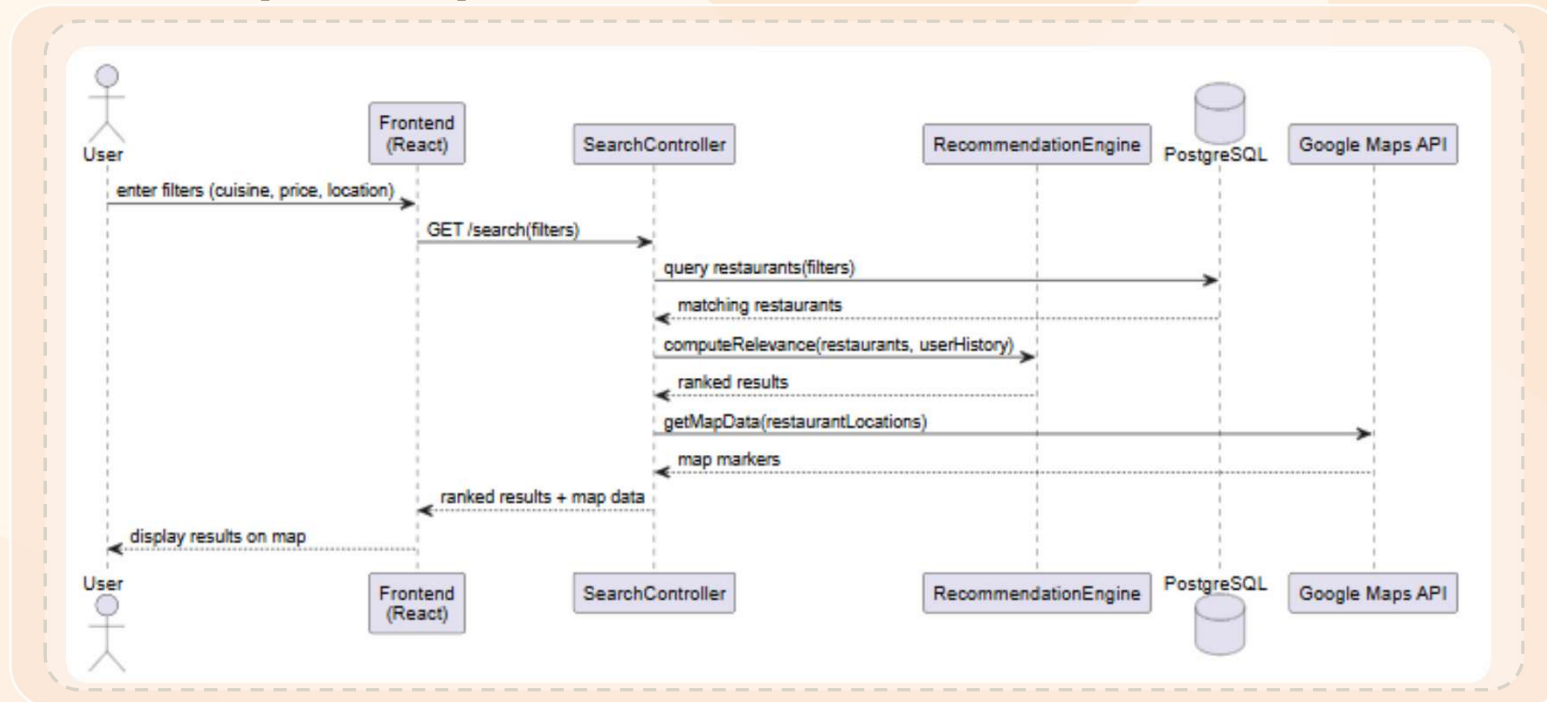
Search & create account

Registered User

All Guest actions + leave reviews & bookmark

Sequence Diagram

Search for Restaurants: a representative operation



1. Enter filters

2. Call SearchController

3. Query PostgreSQL

4. Score with
Recommendation
Engine

5. Add Google Maps
data

6. Show ranked results

Class Diagram

12 classes across four layers

Model

Restaurant, User, Review...

Service

RestaurantService, AuthService...

Controller

SearchController, ReviewController...

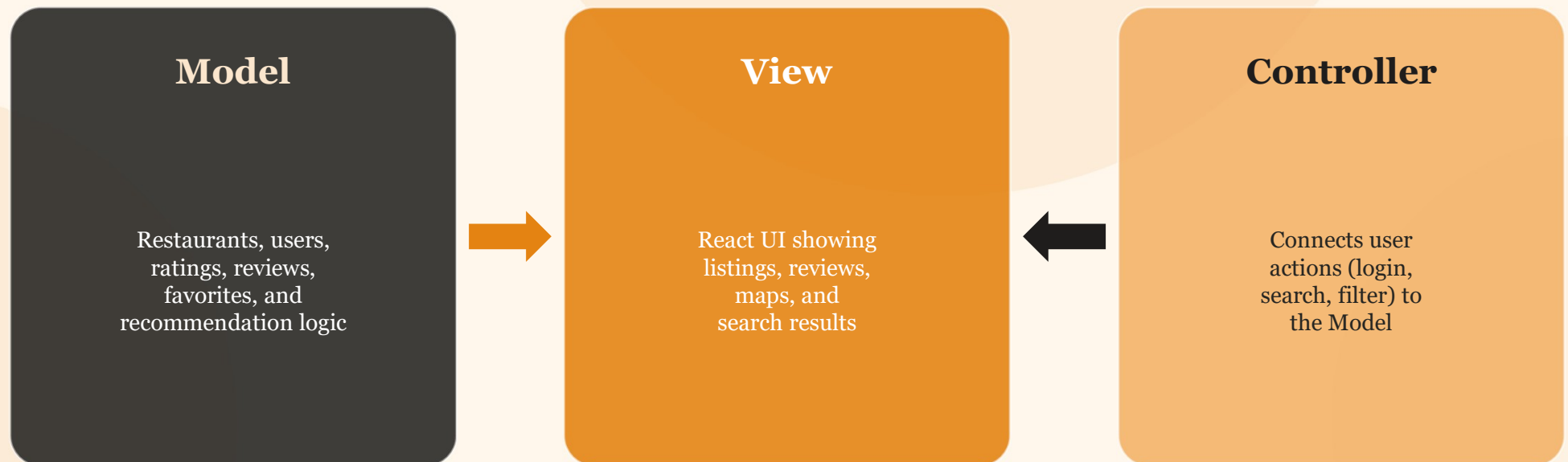
Repository

UserRepository, ReviewRepository...



Architectural Design: MVC

Keeps data, interface, and logic cleanly separated. A good fit for an app with frequent user input and changing content.



Cost & Pricing Estimation

Function Point (FP) Analysis: 122 adjusted function points, 12.2 person-months of effort

\$146,400

Development

\$12,300

Hardware

\$5,521

Software

\$38,333

Personnel Overhead

Total Project Cost

\$202,554

With 20% profit margin

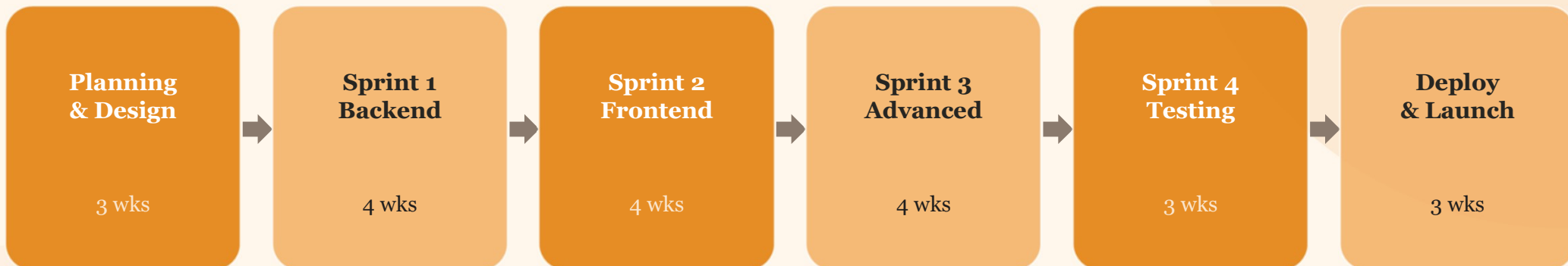
Final Price

\$243,065

Blended rate: \$75/hour

Project Timeline

January 6, 2026 – May 30, 2026 • ~21 weeks • 8 working hours/day, no weekends



Core features (search, accounts, reviews) come first. Advanced features (recommendations, bookmarks, maps) and testing come after.

Comparison with Similar Apps

Yelp

Ranks by review count, same order for everyone

Google Maps

No vibe filter, no focused save/review flow

TripAdvisor

Broad city coverage, but not personalized

How DineMatch is different

- Our own engine scores results per user
- We use Google Maps just for the map. We build the rest ourselves
- Simple features: search, review, bookmark, recommend
- Free open-source tools keep our costs low

Test Case Demo: RecommendationEngine

Testing tool: Jest. Black-box testing with true, false, and boundary cases (9/9 passed)



TRUE CASE (TC1): Perfect Match

Input:

Italian restaurant, price 2, rating 4.5 vs. Italian filter, maxPrice 3

Score = 40 (cuisine) + 30 (price) + 18 (rating) = 88.0

Result: PASS, score matches expected



FALSE CASE (TC2): No Match

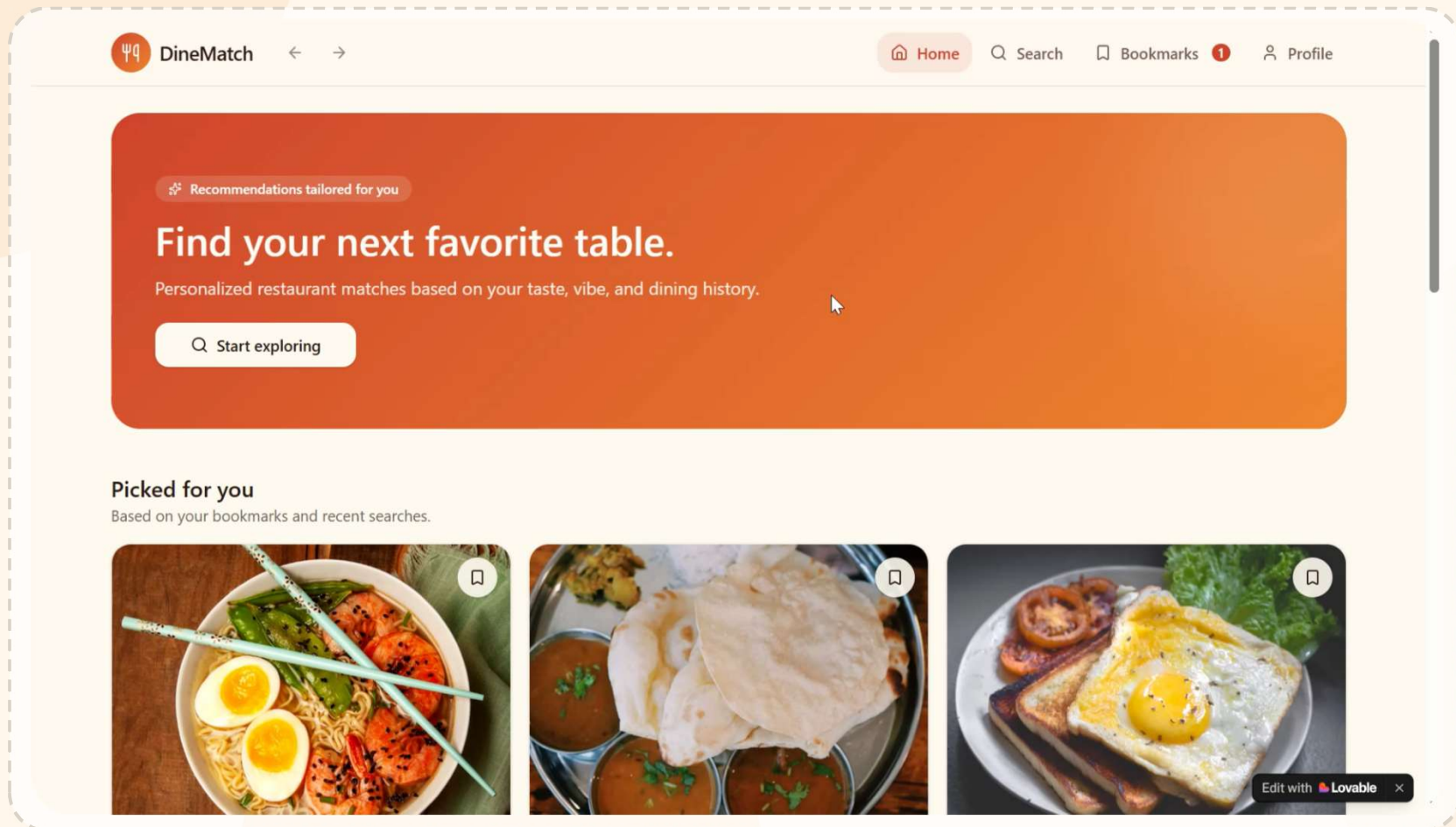
Input:

Mexican restaurant, too far away vs. Italian filter, maxDistance 3km

Raw score = -2, clamped to 0

Result: PASS, score correctly clamped

Prototype



Challenges, Lessons Learned & Conclusion

“I think DineMatch fills a real gap. Most apps show restaurants but don’t personalize results.”

TESTING TOOL

We switched from JUnit to Jest since our scoring logic is shared with React. I think Jest works just as well.

COST METHOD

We used Function Points instead of Application Composition. I think it gave us a more detailed estimate.